

File & Directory

prof.ssa Taffurelli

1. Il File

- Un **file** è una raccolta di dati memorizzati su un disco.
- Ogni file è identificato da:
 - **un nome**
 - **un percorso di directory** (path).
- Quando un file viene aperto per **leggere** o **scrivere** dati, esso viene gestito come uno **stream**.

2. Lo Stream

- Uno **stream** è una **sequenza di byte** che può essere letta o scritta in modo **sequenziale**.
- Rappresenta il **canale di comunicazione** attraverso cui i dati vengono trasferiti tra il programma e il file.

Tipi principali di Stream

Input Stream

- Serve per **leggere i dati** da un file.
- Permette quindi di eseguire operazioni di **lettura**.

Output Stream

- Serve per **scrivere dati** all'interno di un file.
- Permette quindi di eseguire operazioni di **scrittura**.

Il Namespace System.IO

Il namespace **System.IO** mette a disposizione diverse classi per la gestione dei file.

Con queste classi è possibile:

- creare file
- eliminare file
- leggere dati da un file
- scrivere dati in un file
- aprire e chiudere file
- gestire directory

Sr.No.	I/O Class & Description
1	BinaryReader Reads primitive data from a binary stream.
2	BinaryWriter Writes primitive data in binary format.
3	BufferedStream A temporary storage for a stream of bytes.
4	Directory Helps in manipulating a directory structure.
5	DirectoryInfo Used for performing operations on directories.
6	DriveInfo Provides information for the drives.

7	File Helps in manipulating files.
8	FileInfo Used for performing operations on files.
9	FileStream Used to read from and write to any location in a file.
10	MemoryStream Used for random access to streamed data stored in memory.
11	Path Performs operations on path information.
12	StreamReader Used for reading characters from a byte stream.
13	StreamWriter Is used for writing characters to a stream.
14	StringReader Is used for reading from a string buffer.
15	StringWriter Is used for writing into a string buffer.

In C#, per lavorare con le **cartelle del file system**, il namespace **System.IO** mette a disposizione due classi principali:

- **Directory**
- **DirectoryInfo**

Entrambe permettono di eseguire operazioni sulle directory, ma utilizzano approcci diversi.

Classe Directory

La classe **Directory** utilizza **metodi statici**, quindi non è necessario creare un oggetto della classe per usarla.

Serve per eseguire **operazioni rapide e dirette** sulle cartelle.

Esempio: creazione di una directory

```
Directory.CreateDirectory(@"C:\percorso\alla\nuova\cartella");
```

Classe DirectoryInfo

La classe **DirectoryInfo** rappresenta **una specifica directory** del file system.

Permette di:

- ottenere **informazioni dettagliate** sulla directory
- utilizzare **proprietà e metodi** dell'oggetto

È quindi **più flessibile** rispetto alla classe **Directory**.

```
DirectoryInfo di = Directory.CreateDirectory(dirName);
```

```
WriteLine($"Full name: {di.FullName},\n Name: {di.Name},\n Parent: {di.Parent}");
```

- **FullName** → percorso completo della directory
- **Name** → nome della directory
- **Parent** → directory padre

Confronto

Directory	DirectoryInfo
Usa metodi statici	Usa oggetti
Non serve creare istanze	Serve creare un oggetto
Operazioni semplici e veloci	Maggiori informazioni e controllo

Con la classe **Directory** non è possibile accedere direttamente alle proprietà della directory come invece accade con **DirectoryInfo**

La classe File fornisce metodi statici per operare su file, inclusa la creazione, la copia, la rimozione e altro ancora.

	DIRECTORY	FILE
CREAZIONE	Directory.CreateDirectory(path);	File.Create(path)
CANCELLAZIONE	Directory.Delete(path);	File.Delete(path)
SPOSTAMENTO	Directory.Move(sourceDirName, destDirName)	File.Move(sourceDirName, destDirName)
RIDENOMINAZIONE	Directory.Move(OldName, NewName);	File.Move(OldName, NewName)
COPIA	NON SI PUO' FARE DIRETTAMENTE	File.Copy(fileSorg, fileDest,true); Impostando il terzo parametro su true, il file di destinazione verrà sovrascritto se esiste già. Se invece lo imposti su false (valore predefinito), verrà sollevata un'eccezione IOException se il file di destinazione esiste già.
ESISTENZA	Directory.Exists(path)	File.Exists(path);
		File.ReadAllText
		File.WriteAllText

dove path: @"C:\percorso cartella" o file

Gestione dei percorsi e delle directory in C#

1. Ottenere il percorso completo di un file

Per costruire il **percorso completo di un file** si utilizza la classe `Path`, che permette di combinare in modo sicuro directory e nome del file.

```
string dirname = @"C:\temp";  
string filename = "file.txt";  
string fullpath = Path.Combine(dirname, filename);
```

`Path.Combine()` unisce il **percorso della directory** e il **nome del file** creando il **percorso completo**.

2. Ottenere informazioni sulla directory padre

Per ottenere la **directory padre** di un determinato percorso si utilizza il metodo:

```
Directory.GetParent(string path)
```

Questo metodo restituisce un oggetto **DirectoryInfo** che rappresenta la cartella padre

3. Ottenere il nome completo della directory padre

Directory.GetParent(string path).FullName;

La proprietà **FullName** restituisce il **percorso completo** della directory padre.

4. Ottenere o impostare la directory di lavoro corrente

Esistono due modi principali.

Con la proprietà di **Environment**: **Environment.CurrentDirectory**

Questa proprietà:

- restituisce la **directory di lavoro corrente**
- può anche essere **impostata**.

Proprietà:

```
public static string CurrentDirectory { get; set; }
```

Generalmente corrisponde alla **directory in cui si trova il programma in esecuzione**.

5. Ottenere la directory di lavoro corrente dell'applicazione

Directory.GetCurrentDirectory()

Restituisce il **percorso della directory di lavoro corrente** dell'applicazione.

6. Ottenere il nome completo della directory padre della directory corrente

Combinando i metodi precedenti:

Directory.GetParent(Environment.CurrentDirectory).FullName

Questa istruzione restituisce il **percorso completo della directory padre** della directory di lavoro corrente dell'applicazione.

LE CLASSI STREAMREADER E STREAMWRITER

StreamReader reader = new StreamReader (path)

reader.Close(); => l'oggetto deve essere chiuso dopo il suo utilizzo per evitare che il file rimanga bloccato

Quindi è consigliabile usare il blocco **using**. In tal caso l'istruzione **reader.Close();** va omessa

Una volta ottenuto un oggetto StreamReader, per leggere il contenuto:

- **ReadLine()** => per leggere il file riga per riga
- **ReadToEnd ()** => per leggere il file fino alla fine
- **Read()** => per leggere il file carattere per carattere

```
using ( StreamReader sr = new StreamReader("elenco2.txt")){  
while (!sr.EndOfStream)  
{  
    riga = sr.ReadLine(); // Read lines from the file until the end of the file is reached.  
}
```

```
using (StreamReader sr = new StreamReader("elenco2.txt"))
{
    while (sr.Peek() > -1)           //ritorna un intero che rappresenta il carattere successivo da
                                    // leggere oppure -1 se non sono presenti caratteri da
    {                               // leggere
        Console.WriteLine(sr.ReadLine());
    }
}
*****
```

```
using ( StreamReader sr = new StreamReader("elenco2.txt")){
int ch;
    while (ch=sr.Read() != -1)
    {
        Console.WriteLine(ch);
    }
}
*****
```

```
using ( StreamReader sr = new StreamReader("elenco2.txt")){
    string testo= sr.ReadToEnd();
}
}
```

OSSERVAZIONE

per dividere il testo uso la funzione SPLIT

```
using (StreamReader sr = new StreamReader(fullpath1))  
{  
    string testo = sr.ReadToEnd();  
    char[] separators = new char[] { '\n', '\r', ' ' };  
    arr1 = testo.Split(separators, StringSplitOptions.RemoveEmptyEntries);  
}
```

La classe **StreamWriter** viene utilizzata per scrivere dati in un file di testo.
Fa parte del namespace System.IO e utilizza uno stream di output per inviare dati al file.

Costruttore principale

```
StreamWriter writer = new StreamWriter(path, bool append);
```

dove

path → percorso del file su cui scrivere.

append (booleano) → indica come gestire il contenuto del file.

Significato del parametro **append**

true I nuovi dati vengono aggiunti alla fine del file (append).

false Il file viene sovrascritto. Se esiste, il contenuto precedente viene cancellato.

Il valore false è quello di default se il parametro non viene specificato.

Esempio

```
StreamWriter writer = new StreamWriter("file.txt", true);  
    writer.WriteLine("Nuova riga di testo");  
writer.Close();
```

In questo caso:

- il testo viene **aggiunto alla fine del file**
- il contenuto già presente **non viene cancellato**.

-----RISORSE-----

<https://zetcode.com/csharp/directory/>

[https://www.programiz.com/csharp-programming/files#:~:text=Create%20a%20File%20in%20C%23,Create\(pathName\)%3B](https://www.programiz.com/csharp-programming/files#:~:text=Create%20a%20File%20in%20C%23,Create(pathName)%3B)

https://www.w3schools.com/cs/cs_files.php

